

THE CLAUDE BEAST SERIES

# The Claude Beast

Atlassian MCP: Jira & Confluence Integration

---

*Connecting Claude Code directly to your project management and documentation*

# 35+ tools span Jira, Confluence, Compass, and cross-product search



## OFFICIAL SERVER TOOLS

- **13 Jira tools** — create, edit, search (JQL), transition, comment, worklog, metadata
- **11 Confluence tools** — create/update pages, search (CQL), inline & footer comments
- **6 Compass tools** — components, relationships, custom field definitions



## COMMUNITY BONUS TOOLS

- **Agile boards** — list, get board issues
- **Sprint management** — create, update, get sprint issues, list sprints
- **Batch operations** — bulk issue creation
- **Epic linking** — link issues to epics
- **Attachments** — download from issues

# Choose Official for enterprise security, Community for agile power tools

Comparison

## OFFICIAL ROVO MCP

- **Hosting** — Cloud-hosted by Atlassian
- **Auth** — OAuth 2.1 (enterprise-grade)
- **Scope** — Jira + Confluence + Compass
- **Deployment** — Atlassian Cloud only
- **Admin** — Allowlists, audit logs, client restrictions
- **Rate limits** — Plan-based (up to 1,000 req/hr)

## COMMUNITY (sooperset)

- **Hosting** — Self-hosted (Docker / uvx)
- **Auth** — API tokens, PATs, OAuth 2.0
- **Scope** — Jira + Confluence (no Compass)
- **Deployment** — Cloud + Server/Data Center
- **Agile** — Full sprint/board management
- **License** — MIT open-source, 1.2k+ stars

# Three setup paths -- from one-line URL to full Docker deployment

## SETUP PATHS

- Path A: Official Remote MCP (simplest)
- One URL endpoint in .mcp.json
- **OAuth 2.1 via browser** — done in 2 min
- Path B: Community via Docker
- docker pull + configure env vars
- Supports space/project filtering

## KEY CONFIGURATION

- Path C: Community via uvx (no Docker)
- Same env vars, lighter footprint
- Environment Variables:
- CONFLUENCE\_SPACES\_FILTER
- JIRA\_PROJECTS\_FILTER
- READ\_ONLY\_MODE=true (safety)
- MCP\_VERBOSE=true (debugging)

# From context-aware coding to autonomous sprint planning

Use Cases

## DEVELOPER WORKFLOWS

- **Context-Aware Coding** — Pull Jira ticket details while writing code -- acceptance criteria, blockers
- **PRD Generation** — Read Confluence specs + Jira epics, generate comprehensive PRDs, write back to Confluence
- **Auto Documentation** — Create tech design docs, sprint retros, and API docs from ticket data

## TEAM WORKFLOWS

- **Sprint Planning** — List boards, manage sprints, bulk create stories from meeting notes
- **Bug Triage** — Find critical bugs, read details, inspect code, comment with fix analysis
- **Meeting Notes to Tickets** — Parse notes and auto-create/update Jira issues

# Turo Engineering: 30+ minutes of autonomous AI execution with MCP

Case Study

- **Stack** — mcp-atlassian + SQLite database + Figma MCP
- **Innovation** — Persistent planning database for long tasks
- Claude decomposes markdown plans into tasks + dependencies
- Dynamic task selection, progress tracking across sessions
- **Result** — Previously deferred tech debt became feasible
- **Example** — Refactored 166-method monolithic service via AI

## Key Insight

Turo's key innovation: a persistent SQLite planning database that lets Claude decompose plans into tasks and track progress across sessions -- enabling 30+ minutes of autonomous execution.

# Enterprise-ready security with OAuth 2.1, audit logs, and permission inheritance

## OFFICIAL SERVER SECURITY

- **OAuth 2.1** — AI client never sees passwords
- **Permission inheritance** — respects existing Atlassian Cloud permissions for every action
- **Admin controls** — allowlists for MCP clients, audit logs for all AI interactions
- ~40% of monthly active users are enterprise

## COMMUNITY SERVER SECURITY

- **API tokens / PATs** — never commit to git
- Use .env files + .gitignore
- READ\_ONLY\_MODE=true for safety
- SSL verification for Server/Data Center
- Both Servers: Use least privilege principle
- Review high-impact changes before confirming

# Composing Atlassian MCP with GitHub, Sentry, Figma, and Database servers

*Cross-Tool Workflows*

## JIRA + CODE WORKFLOWS

- **Atlassian + GitHub MCP** — "Implement JIRA-456 and create a PR"
- **Atlassian + Sentry MCP** — "Check Sentry for BUG-789 error, update the ticket"

## JIRA + OPERATIONS

- **Atlassian + Figma MCP** — Design to code to Jira ticket for review
- **Atlassian + Database MCP** — Query data, generate incident reports in Confluence automatically
- Each MCP server adds composable capabilities

# Atlassian MCP eliminates context switching and enables autonomous workflows

## IMPACT

- **Eliminates context switching** — stay in your IDE
- **Two paths** — Official (enterprise) vs Community (agile power)
- 35+ tools across Jira, Confluence, and Compass
- Natural language replaces JQL/CQL memorization
- **Bidirectional** — reads AND writes issues, pages, comments

## GETTING STARTED

- Setup in ~15 minutes (official is one URL)
- **Autonomous workflows** — 30+ min AI execution (Turo)
- Composable with GitHub, Figma, Sentry MCPs
- **Enterprise-ready** — OAuth 2.1, audit logs, permissions
- Official Rovo MCP is now GA (early 2026)